Sultan Moulay Slimane University
Polydisciplinary Faculty of Khouribga
Department of Mathematics and Computer Science

Major: Computer Engineering and AI
Module: Big Data Ecosystems

MINI-PROJECT REPORT:

REAL TIME NETWORK INTRUSION DETECTION

(Lambda Architecture: Kafka, Spark Streaming, HDFS, Cassandra)

Author:
[**HMADNA AYMANE**]
Profil GitHub:
`https://github.com/AymaneHmadna`

Academic Year: 2025-2026

Contents

1 Project Objective : 2

2 Detailed Technology Stack : 2

2.1 Apache Kafka (Message Broker) : 2

2.2 Apache Spark Structured Streaming (Processing Engine) : 2

2.3 Cassandra (Hot Storage) : 2

2.4 Hadoop HDFS (Cold Storage) : 3

2.5 Docker and Python : 3

3 Technical Description of Files and Scripts : 3

3.1 Infrastructure and Configuration : 3

3.2 Processing and Ingestion Scripts : 3

3.3 Visualization : 4

4 Execution Workflow : 5

4.1 Infrastructure Deployment : 5

4.2 Kafka Topic Creation : 5

4.3 Cassandra Database Modeling : 6

4.4 Starting the Spark Engine : 8

4.5 Launching the Sniffer (Ingestion) : 8

4.6 Dashboard Initialization : 9

4.7 Visualization in Normal Situation : 11

4.8 Attack Simulation (DDoS) : 11

4.9 Real Time Intrusion Detection : 12

4.10 HDFS Archiving Proof : 13

5 Perspectives and Roadmap : 14

1 Project Objective :

This project aims to design and deploy a complete Big Data architecture capable of ingesting, processing, and visualizing network data in real time to detect cyberattacks (DDoS). It implements a full pipeline ranging from packet sniffing to archiving in a Data Lake (HDFS) and real time monitoring (Cassandra/Streamlit).

2 Detailed Technology Stack :

Each component was selected to address a specific Big Data constraint (Velocity, Volume, Variety).

2.1 Apache Kafka (Message Broker) :

Kafka acts as a high performance buffer. It decouples the ingestion layer (Sniffer) from the processing layer (Spark).

- **Role:** Ingests network packets as "topics".
- **Why Kafka?** Its ability to handle millions of events per second ensures no network packet is lost, even if the Spark processor slows down. We use a partitioned topic for parallel reading.

2.2 Apache Spark Structured Streaming (Processing Engine) :

Spark is the brain of the system. It processes data in quasi real time micro batches.

- **Role:** Consumes the Kafka stream, parses JSON, and applies detection logic (filtering port 6666 and volumetric analysis).
- **Why Spark?** Unlike Hadoop MapReduce (too slow), Spark processes data In-Memory. The *Structured Streaming* module ensures Fault Tolerance and guarantees "Exactly once semantics".

2.3 Cassandra (Hot Storage) :

Column oriented NoSQL database.

- **Role:** Stores alerts and metrics for immediate display on the Dashboard.
- **Why Cassandra?** Optimized for massive write heavy workloads. It allows the Dashboard to query the current system state with millisecond latency.

2.4 Hadoop HDFS (Cold Storage) :

Distributed File System.

- **Role:** Archives all raw attack logs in JSON format.
- **Why HDFS?** Offers unlimited scalability and data redundancy. This "Data Lake" will serve in the future to train Machine Learning models on attack history.

2.5 Docker and Python :

- **Docker:** Ensures service isolation via `docker-compose`.
- **Scapy:** Low level Python library for capturing TCP/IP packets.
- **Streamlit:** Framework for the real time visualization interface.

3 Technical Description of Files and Scripts :

The architecture relies on a set of Python scripts and Docker configuration files. Here is their detailed role:

3.1 Infrastructure and Configuration :

- **docker-compose.yml:** The orchestrator. It defines and deploys 6 interconnected services: Zookeeper, Kafka, Namenode (Hadoop), Datanode, Spark, and Cassandra. It configures virtual networks to allow containers to communicate.
- **Dockerfile:** Defines the custom image used by the Spark container. It starts from a Python 3.9 base and installs Java 17 (required by Spark), along with critical libraries: `pyspark`, `kafka-python`, `cassandra-driver`.
- **hadoop.env:** Contains environment variables injected into Hadoop containers. It configures essential HDFS parameters, such as the Namenode address (`fs.defaultFS`) and disables strict permissions to facilitate development.

3.2 Processing and Ingestion Scripts :

- **real_sniffer.py (Ingestion):** Uses the `Scapy` library to listen to the host machine's network interface. It captures every IP packet, extracts metadata (Source IP, port, size), and sends them to the Kafka topic `network-traffic` as JSON.
- **spark_processor.py (Processing):** The core engine. It uses Spark Structured Streaming to read the Kafka stream continuously. It applies a detection rule (traffic to port 6666 is suspect) and writes results simultaneously to Cassandra (Hot path) and HDFS (Cold path).
- **attacker.py (Simulation):** Offensive script generating a flood of UDP packets targeting port 6666. It simulates a massive DDoS attack to validate system responsiveness.

3.3 Visualization :

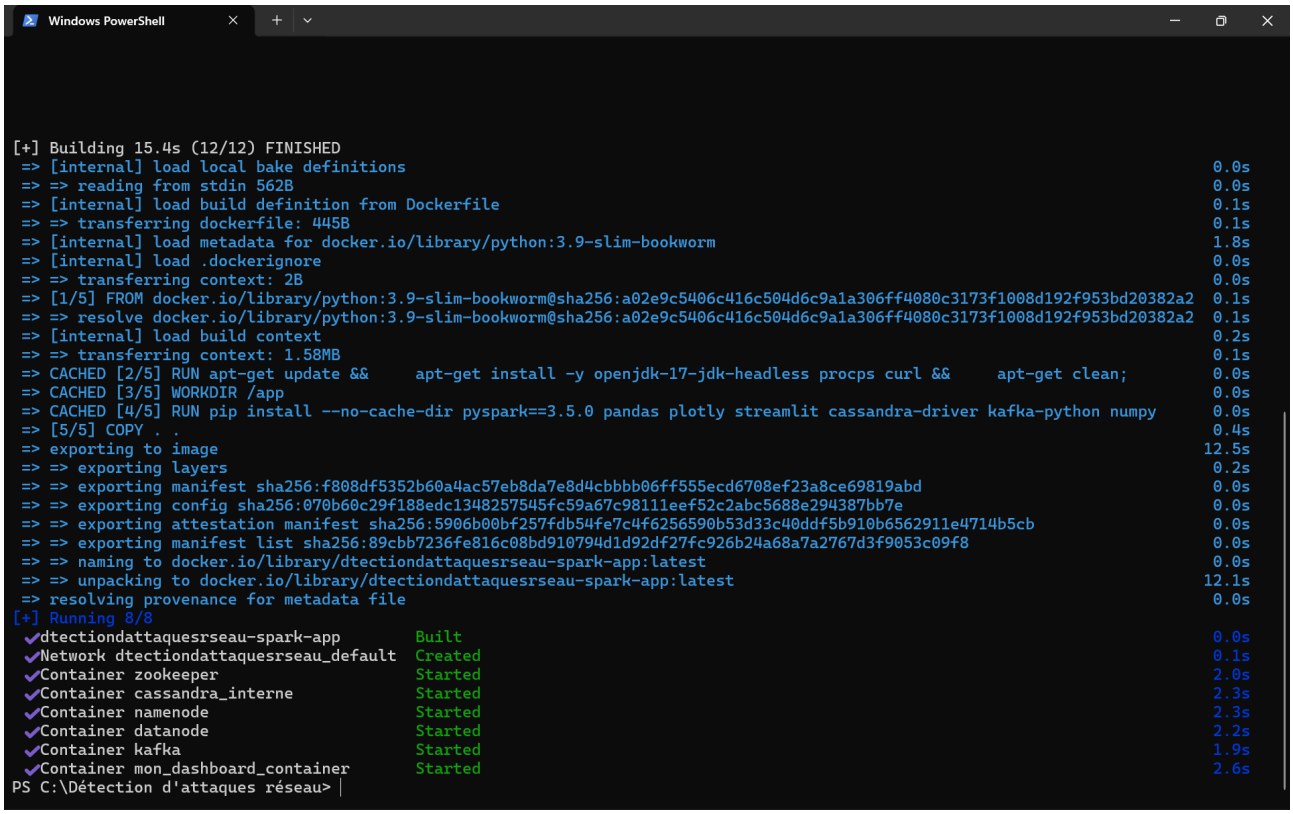
— **app.py (Dashboard):** Application built with `Streamlit`. It queries the Cassandra database every 2 seconds to display KPIs (ex Detected threats) and highlights rows in red if an attack is identified.

4 Execution Workflow :

4.1 Infrastructure Deployment :

Simultaneous launch of all containers via Docker Compose.

```
docker-compose up -d
```



```
Windows PowerShell
[+] Building 15.4s (12/12) FINISHED
 => [internal] load local bake definitions                                                                    0.0s
 => => reading from stdin 562B                                                                                0.0s
 => [internal] load build definition from Dockerfile                                                          0.1s
 => => transferring dockerfile: 445B                                                                          0.1s
 => [internal] load metadata for docker.io/library/python:3.9-slim-bookworm                                   1.8s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim-bookworm@sha256:a02e9c5406c416c504d6c9a1a306ff4080c3173f1008d192f953bd20382a2  0.1s
 => => resolve docker.io/library/python:3.9-slim-bookworm@sha256:a02e9c5406c416c504d6c9a1a306ff4080c3173f1008d192f953bd20382a2  0.1s
 => [internal] load build context                                                                             0.2s
 => => transferring context: 1.58MB                                                                           0.1s
 => CACHED [2/5] RUN apt-get update &&     apt-get install -y openjdk-17-jdk-headless procps curl &&     apt-get clean;  0.0s
 => CACHED [3/5] WORKDIR /app                                                                                 0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir pyspark==3.5.0 pandas plotly streamlit cassandra-driver kafka-python numpy  0.0s
 => [5/5] COPY . .                                                                                            0.4s
 => exporting to image                                                                                       12.5s
 => => exporting layers                                                                                       0.2s
 => => exporting manifest sha256:f808df5352b60a4ac57eb8da7e8d4cbbbb06ff555ecd6708ef23a8ce69819abd             0.0s
 => => exporting config sha256:070b60c29f188edc1348257545fc59a67c98111eef52c2abc5688e294387bb7e               0.0s
 => => exporting attestation manifest sha256:5906b00bf257fdb54fe7c4f6256590b53d33c40ddf5b910b6562911e4714b5cb  0.0s
 => => exporting manifest list sha256:89cbb7236fe816c08bd910794d1d92df27fc926b24a68a7a2767d3f9053c09f8        0.0s
 => => naming to docker.io/library/dtectiondattaquesrseau-spark-app:latest                                    0.0s
 => => unpacking to docker.io/library/dtectiondattaquesrseau-spark-app:latest                                12.1s
 => resolving provenance for metadata file                                                                    0.0s
[+] Running 8/8
 ✔dtectiondattaquesrseau-spark-app        Built                                                              0.0s
 ✔Network dtectiondattaquesrseau_default  Created                                                            0.1s
 ✔Container zookeeper                     Started                                                            2.0s
 ✔Container cassandra_interne             Started                                                            2.3s
 ✔Container namenode                      Started                                                            2.3s
 ✔Container datanode                      Started                                                            2.2s
 ✔Container kafka                         Started                                                            1.9s
 ✔Container mon_dashboard_container       Started                                                            2.6s
PS C:\Détection d'attaques réseau>
```

Figure 1: "Started" status of all services (Kafka, Hadoop, Spark, Cassandra).

4.2 Kafka Topic Creation :

Configuration of the communication channel for network data.

```
docker exec -it kafka kafka-topics --create --topic network-traffic --bootstrap-
    server localhost:9092 --partitions 1 --replication-factor 1
```

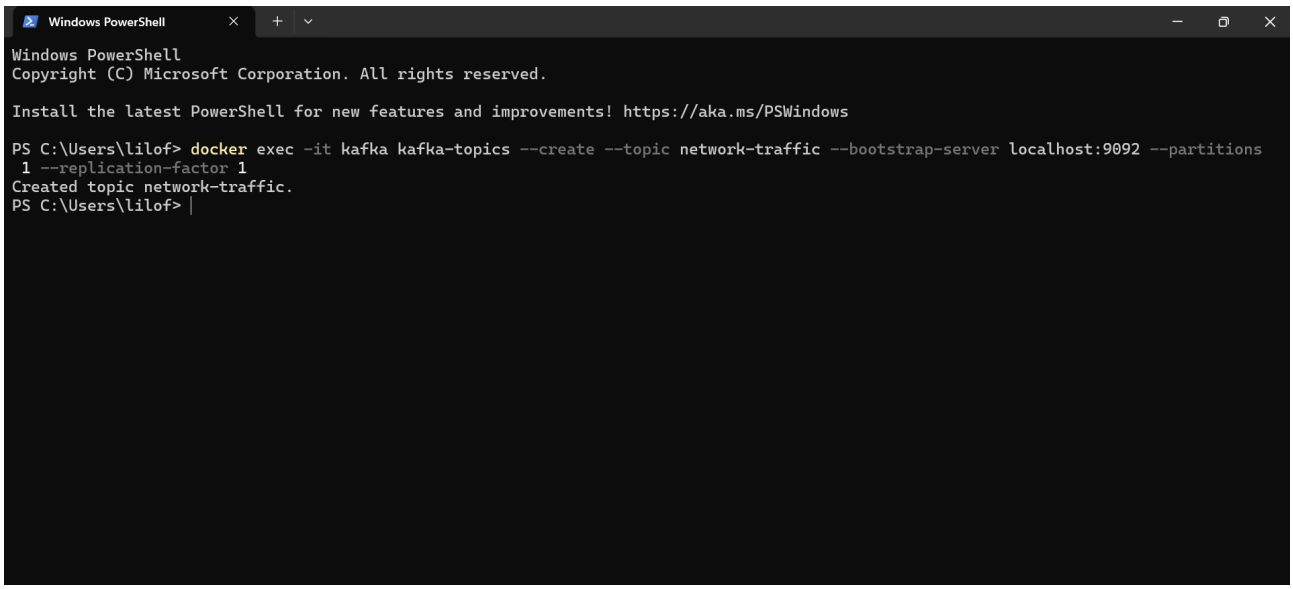


Figure 2: Successful creation of the 'network-traffic' topic.

4.3 Cassandra Database Modeling :

Creation of the Keyspace and the real time storage table.

```
docker exec -it cassandra_interne cqlsh -e "CREATE KEYSPACE IF NOT EXISTS
    projet_logs WITH replication = {'class': 'SimpleStrategy', '
    replication_factor': 1}; CREATE TABLE IF NOT EXISTS projet_logs.resultats_ips
     (ip text PRIMARY KEY, count int, last_seen timestamp, prediction text);"
```

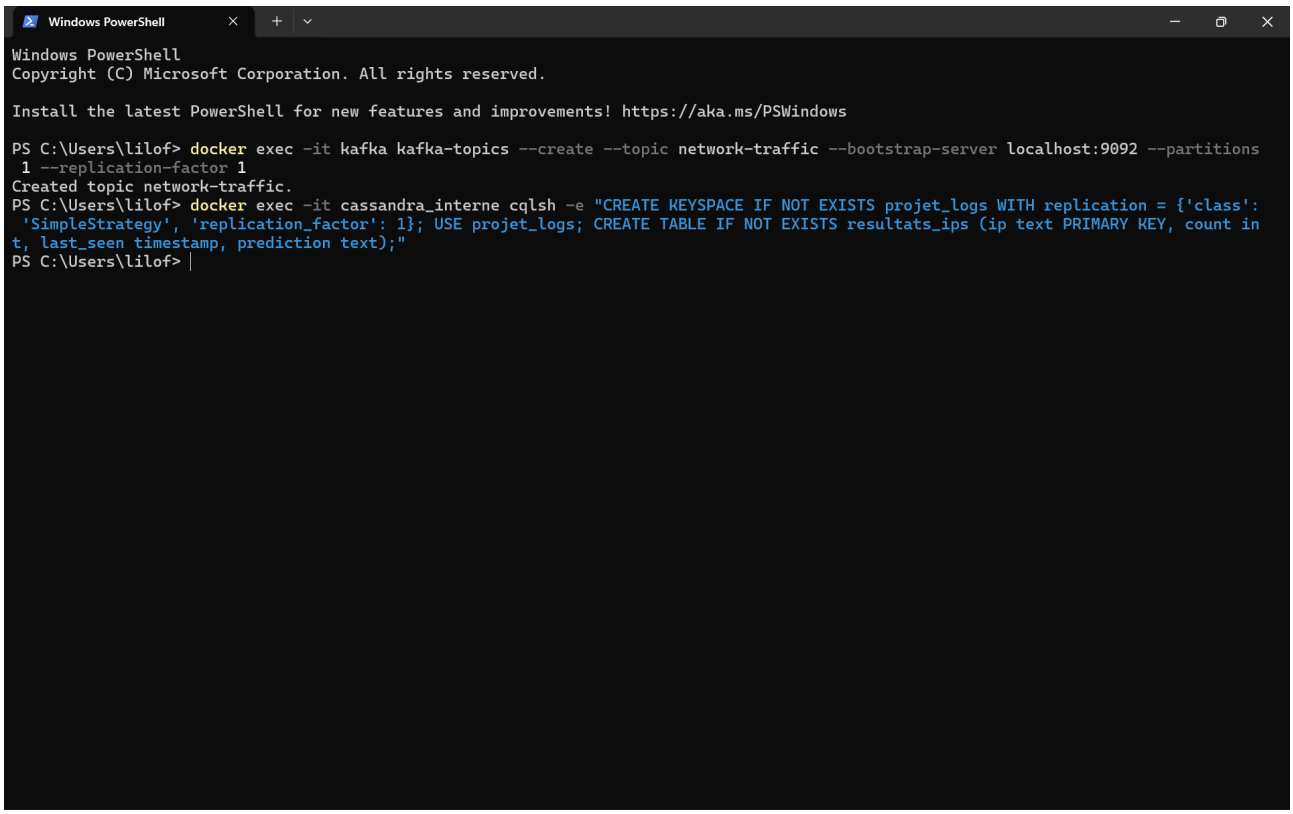
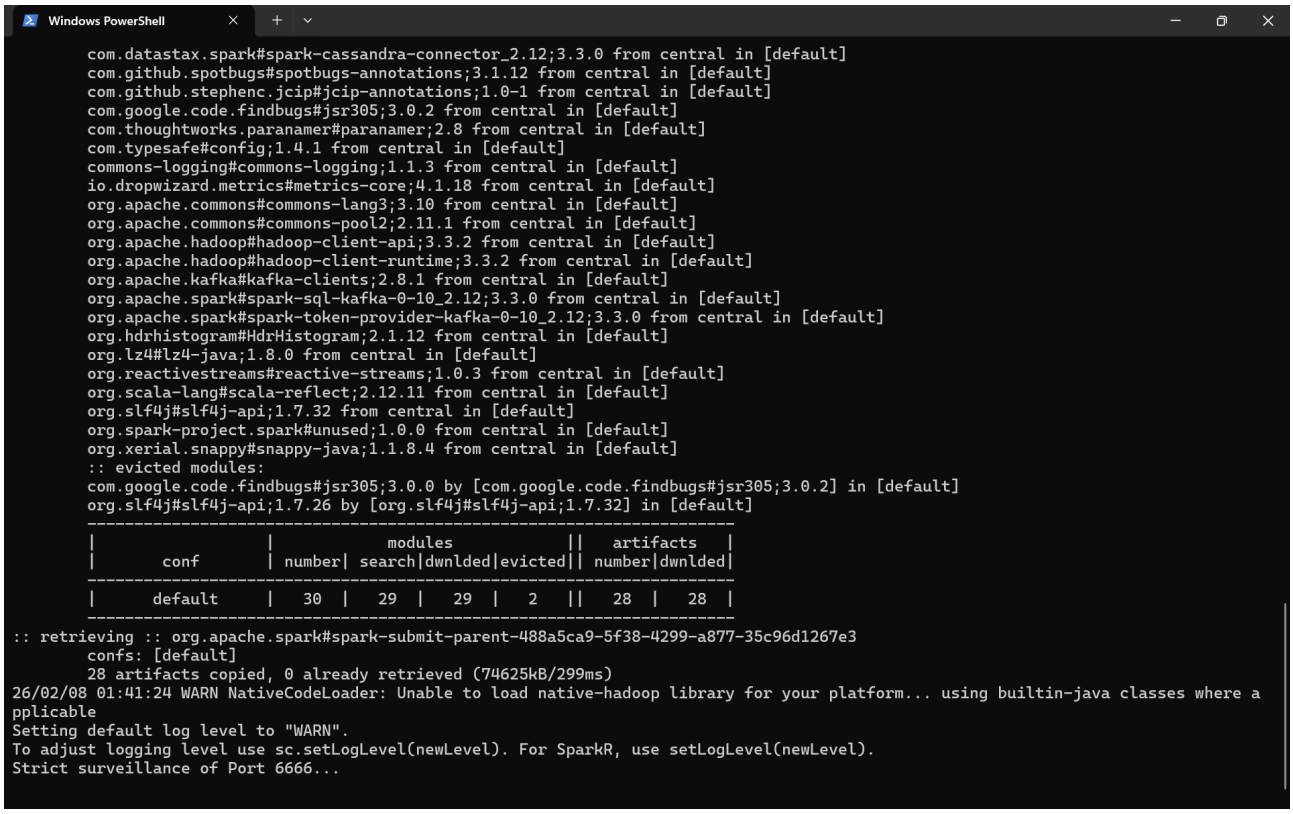


Figure 3: Table structure ready in Cassandra.

4.4 Starting the Spark Engine :

The Spark processor subscribes to Kafka and applies security rules.

```
docker exec -it mon_dashboard_container python spark_processor.py
```



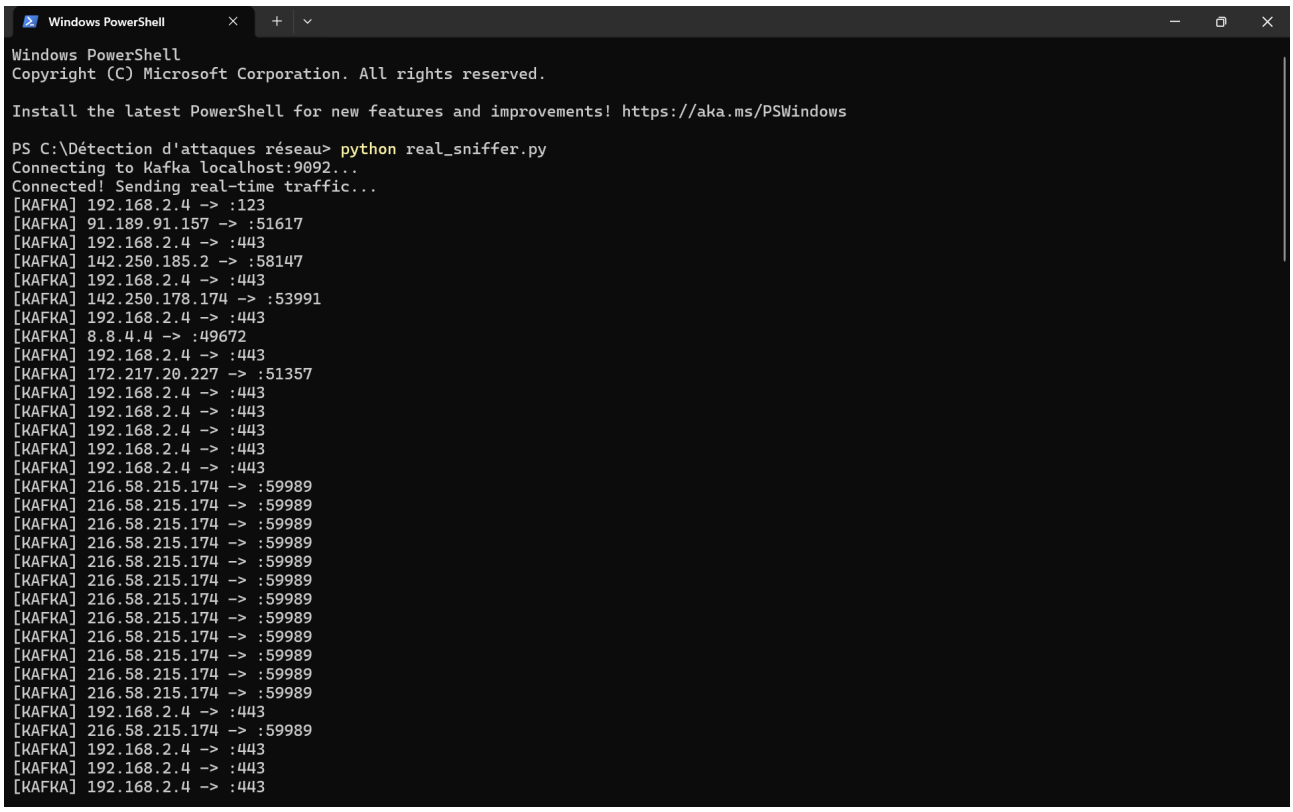
```
Windows PowerShell
        com.datastax.spark#spark-cassandra-connector_2.12;3.3.0 from central in [default]
        com.github.spotbugs#spotbugs-annotations;3.1.12 from central in [default]
        com.github.stephenc.jcip#jcip-annotations;1.0-1 from central in [default]
        com.google.code.findbugs#jsr305;3.0.2 from central in [default]
        com.thoughtworks.paranamer#paranamer;2.8 from central in [default]
        com.typesafe#config;1.4.1 from central in [default]
        commons-logging#commons-logging;1.1.3 from central in [default]
        io.dropwizard.metrics#metrics-core;4.1.18 from central in [default]
        org.apache.commons#commons-lang3;3.10 from central in [default]
        org.apache.commons#commons-pool2;2.11.1 from central in [default]
        org.apache.hadoop#hadoop-client-api;3.3.2 from central in [default]
        org.apache.hadoop#hadoop-client-runtime;3.3.2 from central in [default]
        org.apache.kafka#kafka-clients;2.8.1 from central in [default]
        org.apache.spark#spark-sql-kafka-0-10_2.12;3.3.0 from central in [default]
        org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.3.0 from central in [default]
        org.hdrhistogram#HdrHistogram;2.1.12 from central in [default]
        org.lz4#lz4-java;1.8.0 from central in [default]
        org.reactivestreams#reactive-streams;1.0.3 from central in [default]
        org.scala-lang#scala-reflect;2.12.11 from central in [default]
        org.slf4j#slf4j-api;1.7.32 from central in [default]
        org.spark-project.spark#unused;1.0.0 from central in [default]
        org.xerial.snappy#snappy-java;1.1.8.4 from central in [default]
        :: evicted modules:
        com.google.code.findbugs#jsr305;3.0.0 by [com.google.code.findbugs#jsr305;3.0.2] in [default]
        org.slf4j#slf4j-api;1.7.26 by [org.slf4j#slf4j-api;1.7.32] in [default]
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   30  |   29  |   29  |   2   ||   28  |   28  |
        ---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-488a5ca9-5f38-4299-a877-35c96d1267e3
        confs: [default]
        28 artifacts copied, 0 already retrieved (74625kB/299ms)
26/02/08 01:41:24 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where a
pplicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Strict surveillance of Port 6666...
```

Figure 4: Spark listening: "Strict surveillance of Port 6666".

4.5 Launching the Sniffer (Ingestion) :

Capturing real network traffic and sending it to the pipeline.

```
python real_sniffer.py
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Détection d'attaques réseau> python real_sniffer.py
Connecting to Kafka localhost:9092...
Connected! Sending real-time traffic...
[KAFKA] 192.168.2.4 -> :123
[KAFKA] 91.189.91.157 -> :51617
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 142.250.185.2 -> :58147
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 142.250.178.174 -> :53991
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 8.8.4.4 -> :49672
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 172.217.20.227 -> :51357
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 216.58.215.174 -> :59989
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 192.168.2.4 -> :443
[KAFKA] 192.168.2.4 -> :443
```

Figure 5: The sniffer injects captured packets into Kafka.

4.6 Dashboard Initialization :

Starting the web supervision interface.

```
docker exec -it mon_dashboard_container streamlit run app.py
```

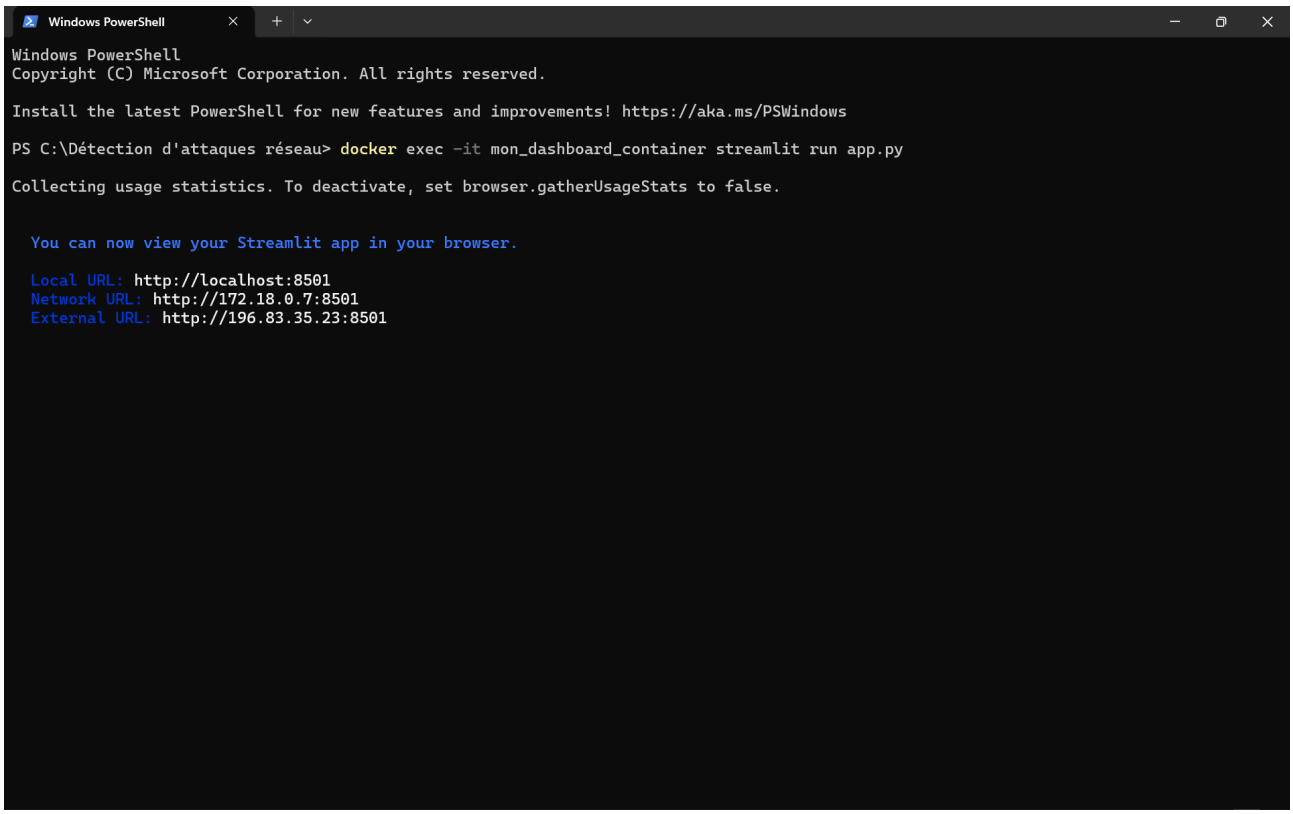


Figure 6: Application accessible on port 8501.

4.7 Visualization in Normal Situation :

The network is calm, the system displays a healthy status.

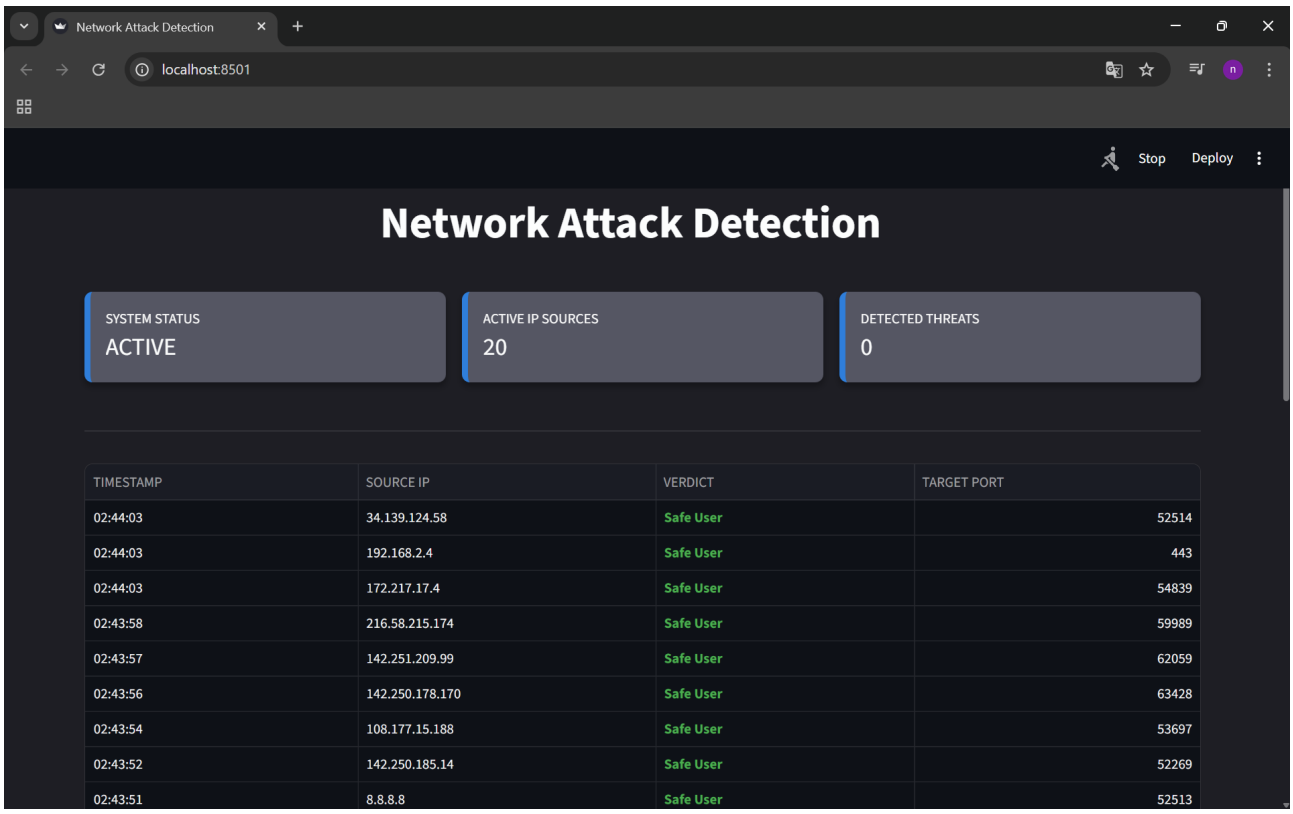
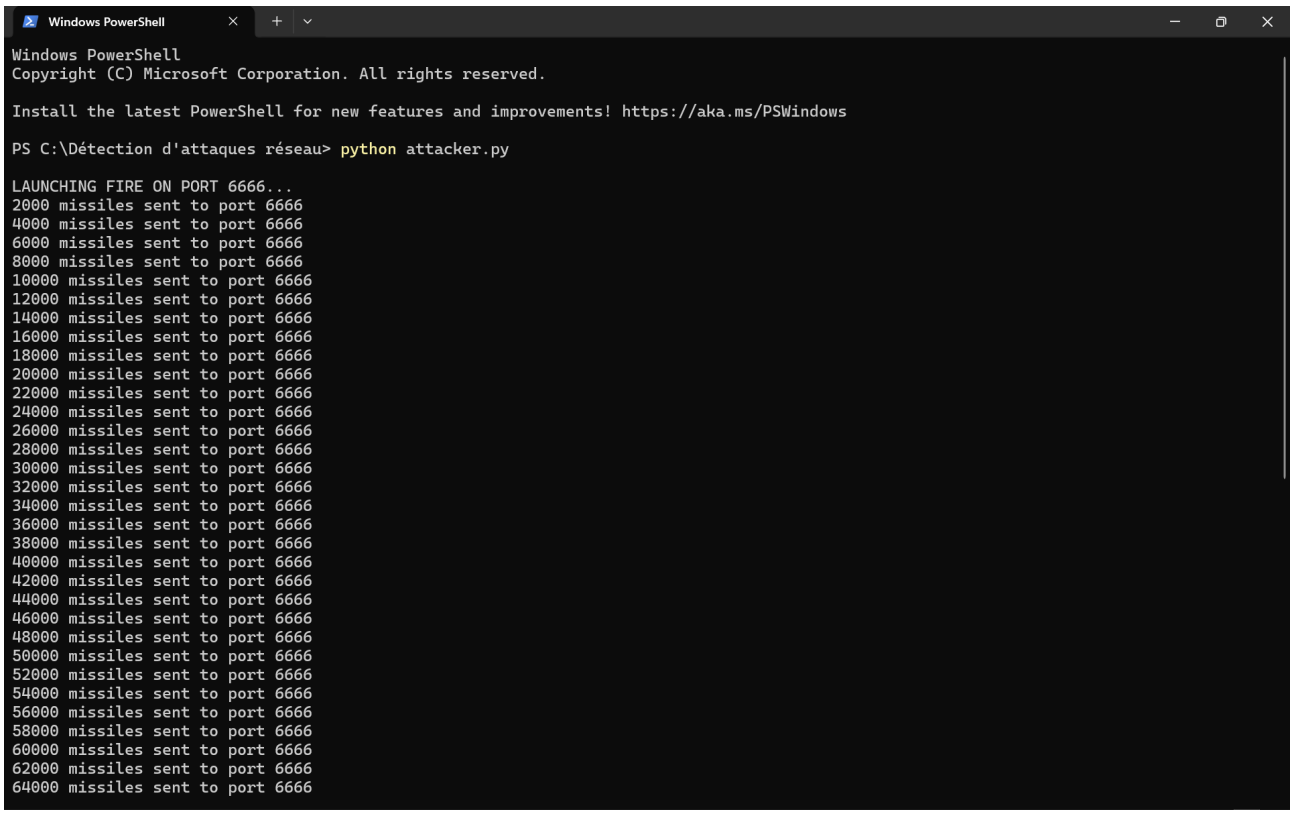


Figure 7: Green Dashboard: "Safe User" verdict for legitimate traffic.

4.8 Attack Simulation (DDoS) :

Running the offensive script generating thousands of packets.

```
python attacker.py
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Détection d'attaques réseau> python attacker.py

LAUNCHING FIRE ON PORT 6666...
2000 missiles sent to port 6666
4000 missiles sent to port 6666
6000 missiles sent to port 6666
8000 missiles sent to port 6666
10000 missiles sent to port 6666
12000 missiles sent to port 6666
14000 missiles sent to port 6666
16000 missiles sent to port 6666
18000 missiles sent to port 6666
20000 missiles sent to port 6666
22000 missiles sent to port 6666
24000 missiles sent to port 6666
26000 missiles sent to port 6666
28000 missiles sent to port 6666
30000 missiles sent to port 6666
32000 missiles sent to port 6666
34000 missiles sent to port 6666
36000 missiles sent to port 6666
38000 missiles sent to port 6666
40000 missiles sent to port 6666
42000 missiles sent to port 6666
44000 missiles sent to port 6666
46000 missiles sent to port 6666
48000 missiles sent to port 6666
50000 missiles sent to port 6666
52000 missiles sent to port 6666
54000 missiles sent to port 6666
56000 missiles sent to port 6666
58000 missiles sent to port 6666
60000 missiles sent to port 6666
62000 missiles sent to port 6666
64000 missiles sent to port 6666
```

Figure 8: Massive packet flood targeting port 6666.

4.9 Real Time Intrusion Detection :

Spark instantly detects the anomaly. The Dashboard turns Red.

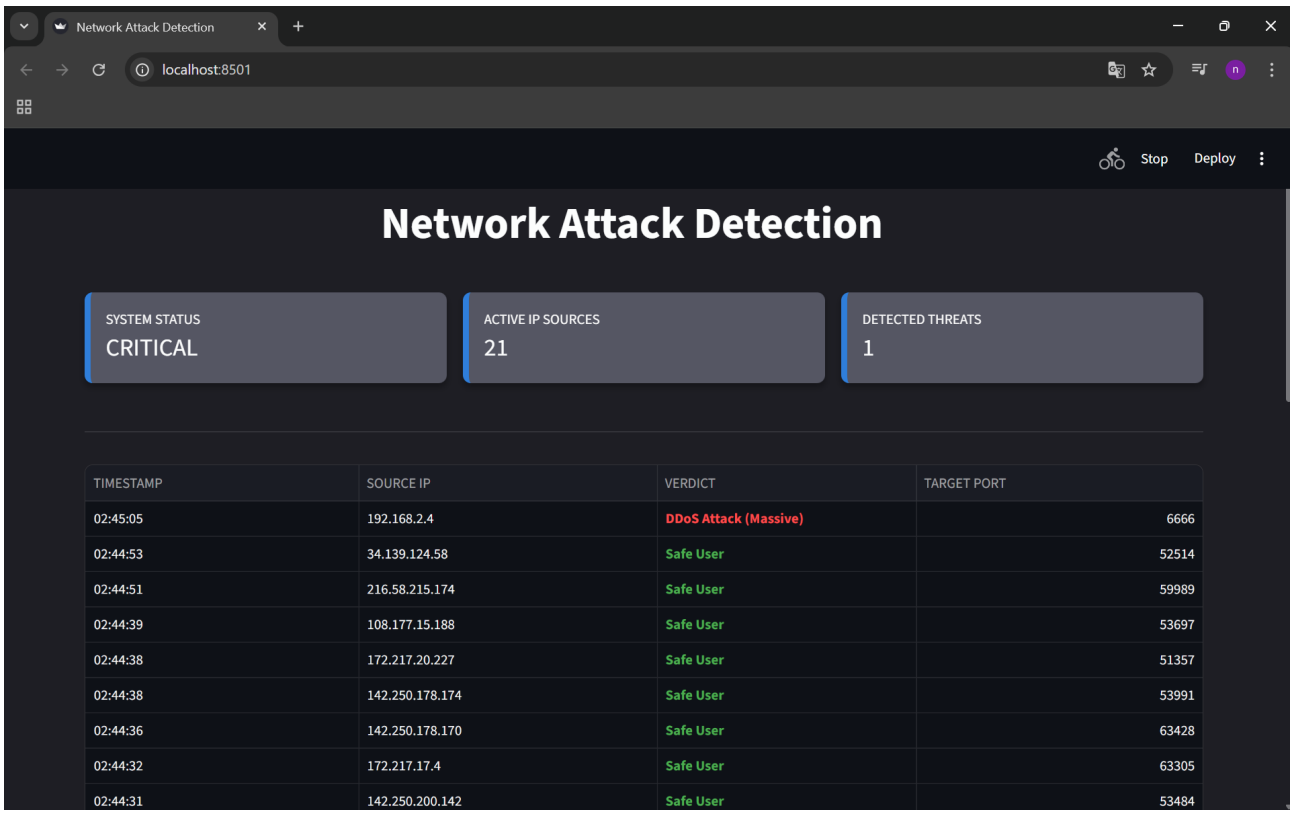
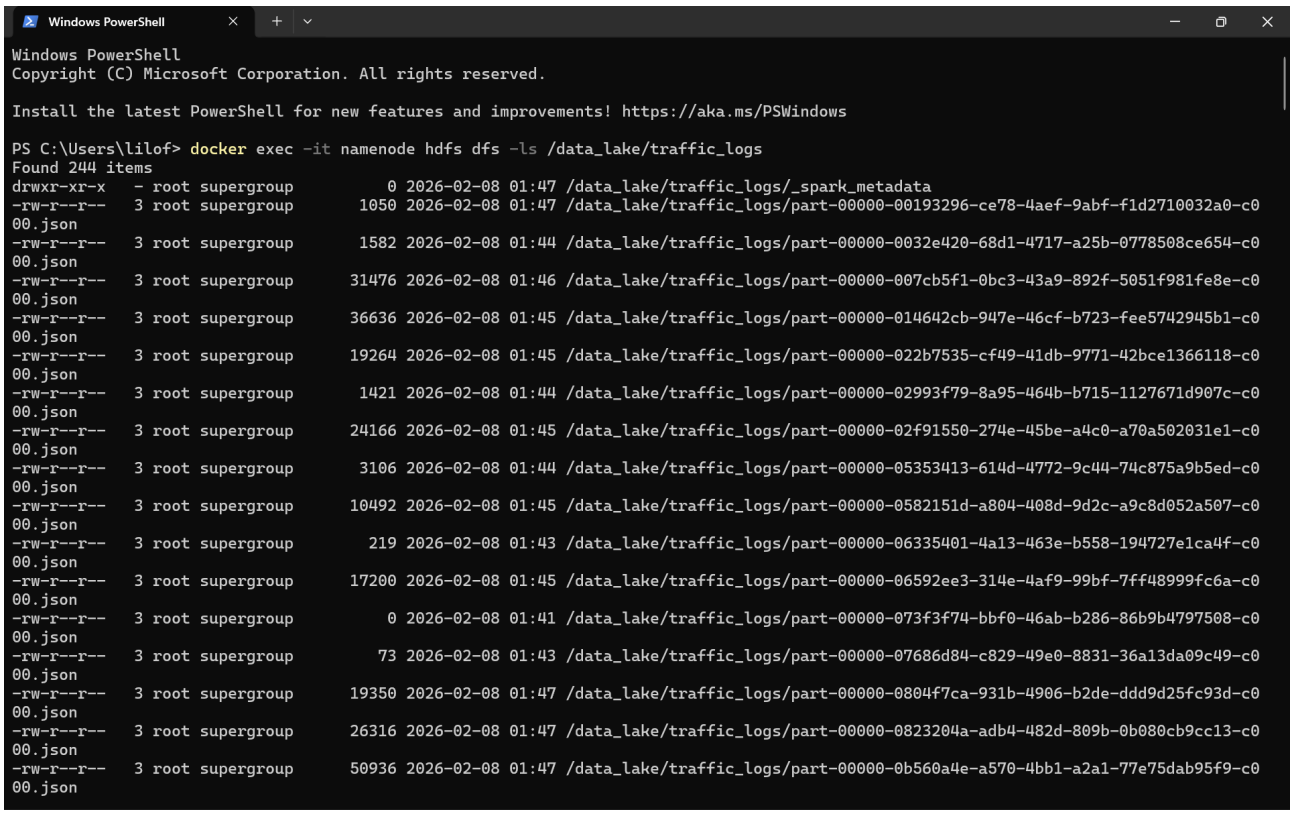


Figure 9: ALERT: The DDoS attack is identified and displayed.

4.10 HDFS Archiving Proof :

Verification that logs have been securely saved in the Data Lake.

```
docker exec -it namenode hdfs dfs -ls /data_lake/traffic_logs
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\lilof> docker exec -it namenode hdfs dfs -ls /data_lake/traffic_logs
Found 244 items
drwxr-xr-x   - root supergroup          0 2026-02-08 01:47 /data_lake/traffic_logs/_spark_metadata
-rw-r--r--   3 root supergroup       1050 2026-02-08 01:47 /data_lake/traffic_logs/part-00000-00193296-ce78-4aef-9abf-f1d2710032a0-c0
00.json
-rw-r--r--   3 root supergroup       1582 2026-02-08 01:44 /data_lake/traffic_logs/part-00000-0032e420-68d1-4717-a25b-0778508ce654-c0
00.json
-rw-r--r--   3 root supergroup      31476 2026-02-08 01:46 /data_lake/traffic_logs/part-00000-007cb5f1-0bc3-43a9-892f-5051f981fe8e-c0
00.json
-rw-r--r--   3 root supergroup      36636 2026-02-08 01:45 /data_lake/traffic_logs/part-00000-014642cb-947e-46cf-b723-fee5742945b1-c0
00.json
-rw-r--r--   3 root supergroup      19264 2026-02-08 01:45 /data_lake/traffic_logs/part-00000-022b7535-cf49-41db-9771-42bce1366118-c0
00.json
-rw-r--r--   3 root supergroup       1421 2026-02-08 01:44 /data_lake/traffic_logs/part-00000-02993f79-8a95-464b-b715-1127671d907c-c0
00.json
-rw-r--r--   3 root supergroup      24166 2026-02-08 01:45 /data_lake/traffic_logs/part-00000-02f91550-274e-45be-a4c0-a70a502031e1-c0
00.json
-rw-r--r--   3 root supergroup       3106 2026-02-08 01:44 /data_lake/traffic_logs/part-00000-05353413-614d-4772-9c44-74c875a9b5ed-c0
00.json
-rw-r--r--   3 root supergroup      10492 2026-02-08 01:45 /data_lake/traffic_logs/part-00000-0582151d-a804-408d-9d2c-a9c8d052a507-c0
00.json
-rw-r--r--   3 root supergroup        219 2026-02-08 01:43 /data_lake/traffic_logs/part-00000-06335401-4a13-463e-b558-194727e1ca4f-c0
00.json
-rw-r--r--   3 root supergroup      17200 2026-02-08 01:45 /data_lake/traffic_logs/part-00000-06592ee3-314e-4af9-99bf-7ff48999fc6a-c0
00.json
-rw-r--r--   3 root supergroup          0 2026-02-08 01:41 /data_lake/traffic_logs/part-00000-073f3f74-bbf0-46ab-b286-86b9b4797508-c0
00.json
-rw-r--r--   3 root supergroup         73 2026-02-08 01:43 /data_lake/traffic_logs/part-00000-07686d84-c829-49e0-8831-36a13da09c49-c0
00.json
-rw-r--r--   3 root supergroup      19350 2026-02-08 01:47 /data_lake/traffic_logs/part-00000-0804f7ca-931b-4906-b2de-ddd9d25fc93d-c0
00.json
-rw-r--r--   3 root supergroup      26316 2026-02-08 01:47 /data_lake/traffic_logs/part-00000-0823204a-adb4-482d-809b-0b080cb9cc13-c0
00.json
-rw-r--r--   3 root supergroup      50936 2026-02-08 01:47 /data_lake/traffic_logs/part-00000-0b560a4e-a570-4bb1-a2a1-77e75dab95f9-c0
00.json
```

Figure 10: JSON files durably stored in Hadoop.

5 Perspectives and Roadmap :

A major evolution planned for this project was the integration of Unsupervised Machine Learning via the **K-Means** algorithm. The goal was to move beyond static rules to detect unknown behavioral anomalies through clustering.

However, this feature could not be deployed in production in the final version. Critical compatibility conflicts arose between `Spark MLlib` libraries, Scala dependencies, and the containerized Hadoop environment, causing instabilities in the streaming pipeline. Given the strict deadline constraints, the engineering decision was to prioritize the robustness of the current Lambda Architecture (guaranteeing flawless ingestion and storage) and postpone the ML module integration to a future iteration.